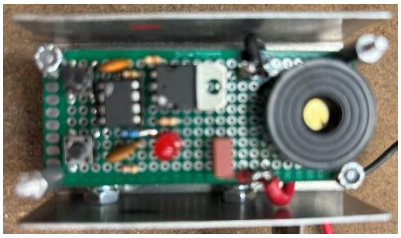
A Chevy Bolt EV 12V Battery Monitor

By R.G. Sparber

Scope



This device continuously monitors the 12-volt battery in a Chevy Bolt EV (electric car) and triggers an audible and visual alarm when the voltage drops below a user-defined threshold. This threshold is set just above the point where the car's computers start to fail randomly.

The cost of materials is under $10. The fun of designing and building it was priceless.

Background

When the 12V battery in an internal combustion engine car starts to fail, you can tell because it becomes difficult to start.

My battery-powered electric car, a 2023 Chevy Bolt EV, has a 12V battery that powers all the car's computers, plus a 400V battery that powers the traction motor. The 400V battery also powers a converter, which charges the 12V battery.

I can only speak to my experience with this car – when its 12V battery nears end-of-life, I get random and rare error messages and temporary functionality issues.

When I took my car to the dealer, their first question was the age of the 12V battery. Sure enough, the battery failed the load test.

Chevy did an amazing job of designing the monitoring and caring of the 400-volt traction battery in my car. I have no idea why they totally ignored the 12V battery.

I needed to add a system to my car that monitored for a weak 12V battery.

Contents

Scope 1
Background 1
Know Versus Educated Guess 3
Details of the Problem 3
Existing Solutions 4
My Solution 5
High-Level Design 6
Hardware Design 7
    Alarm Strategy 8
    Circuit Design Choice 9
    High-Level Circuit Description 10
Detailed Circuit Analysis 12
Parts List 14
Software Overview 16
Hardware Registers 17
    ADMUX 17
    CLKPR 18
    EEPROM 18
Software Middle Level View 19
    setup() 19
    loop() 19
Software Low-Level View 19
What Next? 20

Know Versus Educated Guess

I am confident that a low battery voltage will cause sub-system computers to generate error messages. This condition is commonly called a brownout.

Reading about my particular battery, an AGM[1] type, I believe that 12V is a reasonable threshold for performing a load test. It should be low enough to avoid false alarms yet high enough to tell me before computers start to fail.

What I don't know is how the low-voltage condition behaves over time. I assume that very short dips in voltage are filtered out by each computer's power supply, but at some point, the computer will experience random errors.

I now have a new AGM battery in my car with my monitor set up to alarm when the voltage drops below 12.00 volts. I have no simple way to simulate a bad battery, so I will have to wait until the battery reaches end-of-life to know if my circuit properly warns me before I see random computer errors.

Details of the Problem

My car will run on a battery that fails the load test, but intermittent computer failures are likely. Eventually, the battery is so bad that the car stops running. Experience has shown that testing the battery with a load tester every 6 months is insufficient for a 3-year-old battery here in Phoenix[2].

The key parameter is the minimum battery voltage. If it dips below 12.0V[3], we are about to have computer problems. It is sufficient to alarm if this condition persists for more than a few milliseconds. I would then run a load test. If there were too many false alarms, I would raise the voltage threshold.

[1] https://lifelinebatteries.com/agm-batteries/
[2] Our high temperatures greatly shorten the life of a lead acid battery.
[3] See https://lifelinebatteries.com/agm-batteries/ and https://www.renogy.com/blogs/general-solar/agm-battery-voltage?srsltid=AfmBOoqL_YV46k2luOsPwY0faNFDDtutEGSxSbZh6mJ9URswvNTE9ePP

Existing Solutions

You can connect a voltmeter to your battery, but it will only tell you what is going on now. The voltage may have dipped low enough to cause a computer to falter in the past, but now it is high enough not to cause a problem.

You can buy a product called an Ancel BM200 Car Battery Monitor. According to their website[4], the device records the voltage every 2 minutes when the car is not being started and every 100 milliseconds during cranking.

Since my car doesn't have a starter, I assume the device records every 2 minutes. This sample rate may not be often enough to catch a momentary brownout that crashes one of the computers. I have no data on this failure mode.

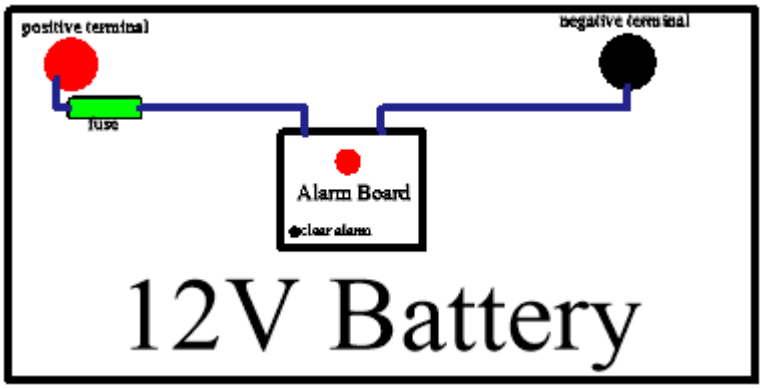
My design checks continuously, then runs a 50-millisecond scan to minimize false alarms. Here, I assume the car's computers can tolerate a single dip in voltage, but a sustained brownout would cause problems.

I'm also concerned about the dependence of any commercial device on an app. All too often, such devices become useless when the company stops supporting the app.

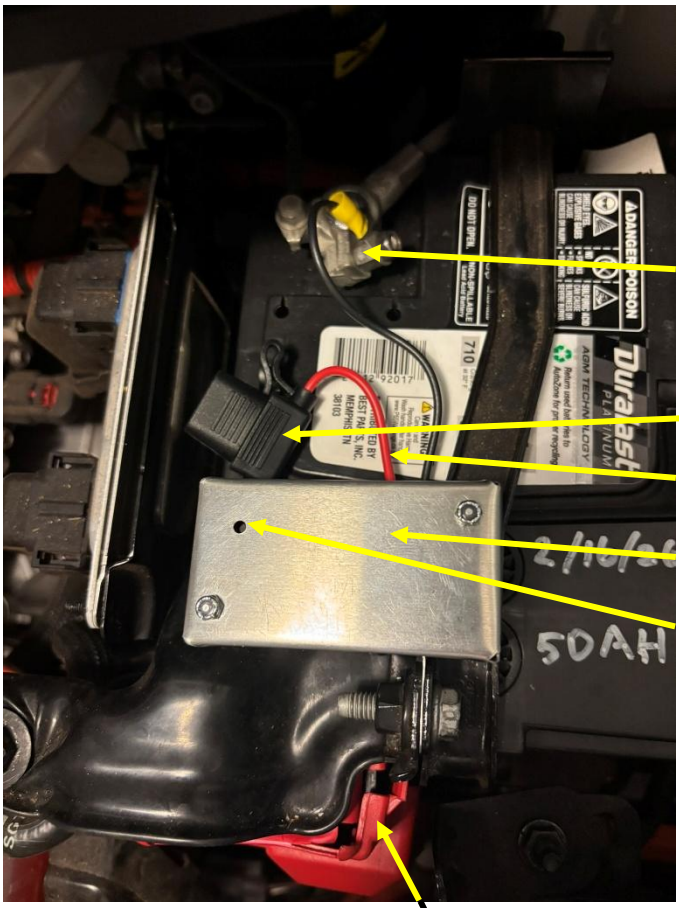
Of course, my biggest problem with any store-bought device is that I learn nothing from buying it. The monitor I designed and built for about 1/3 the price of the BM200 was challenging, fun, educational, and matched exactly what I wanted. Even if it cost more than the store-bought device, I would still make my own monitor.

[4] https://www.ancel.com/products/ancel-bm200

My Solution



I have designed, built, tested, and installed a circuit in my EV that continuously monitors the minimum terminal voltage of the 12V battery. If the voltage drops below 12.0 volts, we measure it 5 more times at 10 millisecond intervals. If the majority of these readings is below this threshold, an LED flashes and the beeper sounds until the circuit is reset by either pressing the RESET button or removing power.



Negative terminal

In-line 3 amp fast blow fuse

Positive terminal wire

Monitoring computer inside metal box

Reset button hole



Positive terminal under the red plastic cover

High-Level Design

Power and signal come directly from the 12V battery. Although the circuit draws no more than 16 milliamperes, we must use a fuse as close to the positive battery terminal as possible to avoid a fire in the event of a fault.

I use an ATTiny85 processor running a program that uses 2312 bytes of program storage and 13 bytes of global variable space. It might seem like overkill to use a processor for such a simple task, but keep in mind that the ATTiny85 costs around $3 and is in an 8-pin package. In quantity and smaller packages, the cost is below $2.

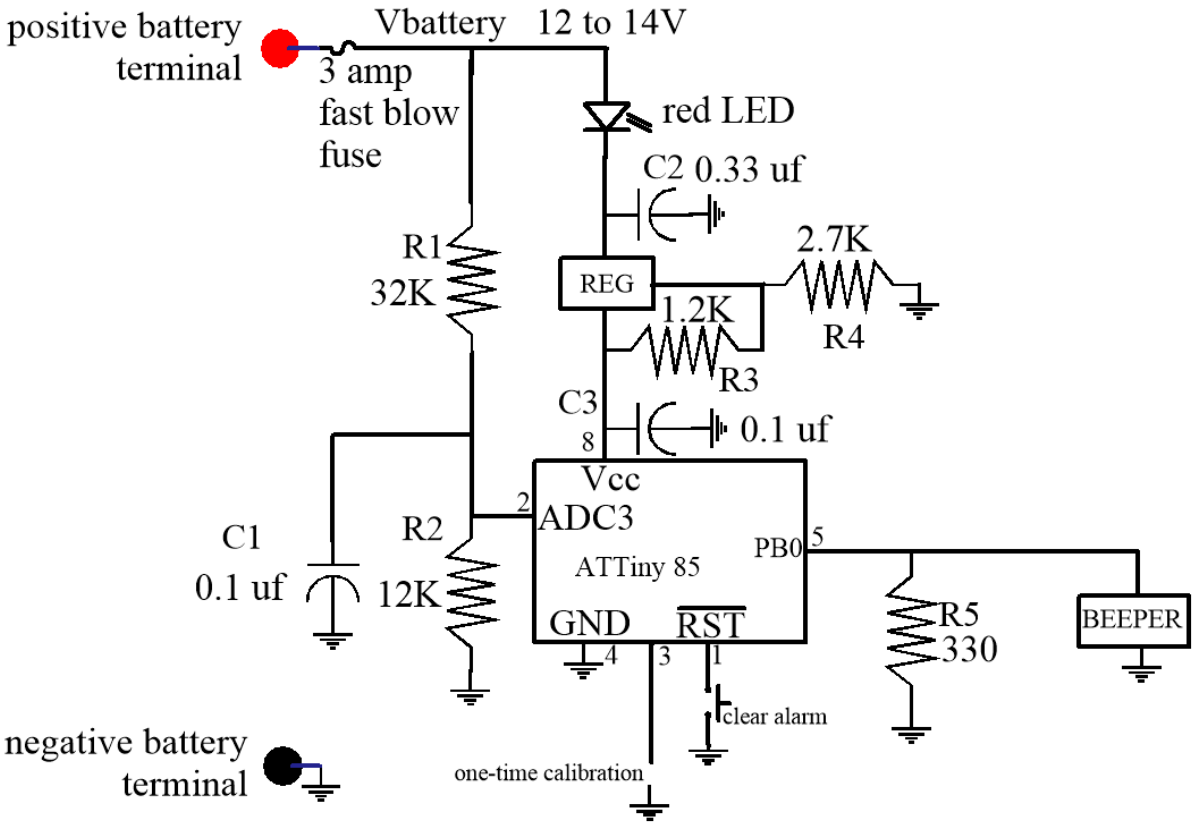
The circuit has three sources of error: resistors, ADC offset, and the voltage reference. The majority of the deviation from nominal is evident during circuit assembly. They are also sensitive to temperature. I use a two-step strategy to minimize these sources of error: calibration and temperature-compensated threshold.

After I assembled the circuit, I powered it from 12.000 volts. Then I pushed the calibration button hidden inside the enclosure. Calibration caused the system to measure the applied battery voltage and record it as the battery threshold voltage in non-volatile memory (EEPROM). This action nulled out all errors at the current temperature.

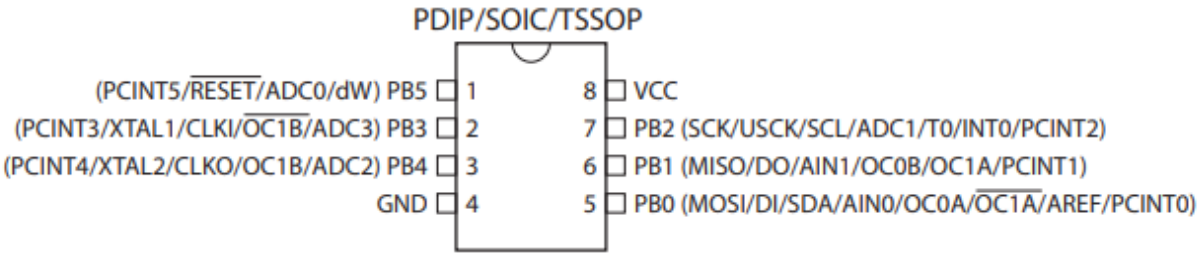
The system then recorded the ambient temperature in EEPROM for later use in creating a temperature-compensated voltage threshold.

During normal operation, the circuit continuously measures the applied battery and the ambient temperature. It uses these parameters to determine if the battery voltage is below the temperature-compensated voltage threshold.

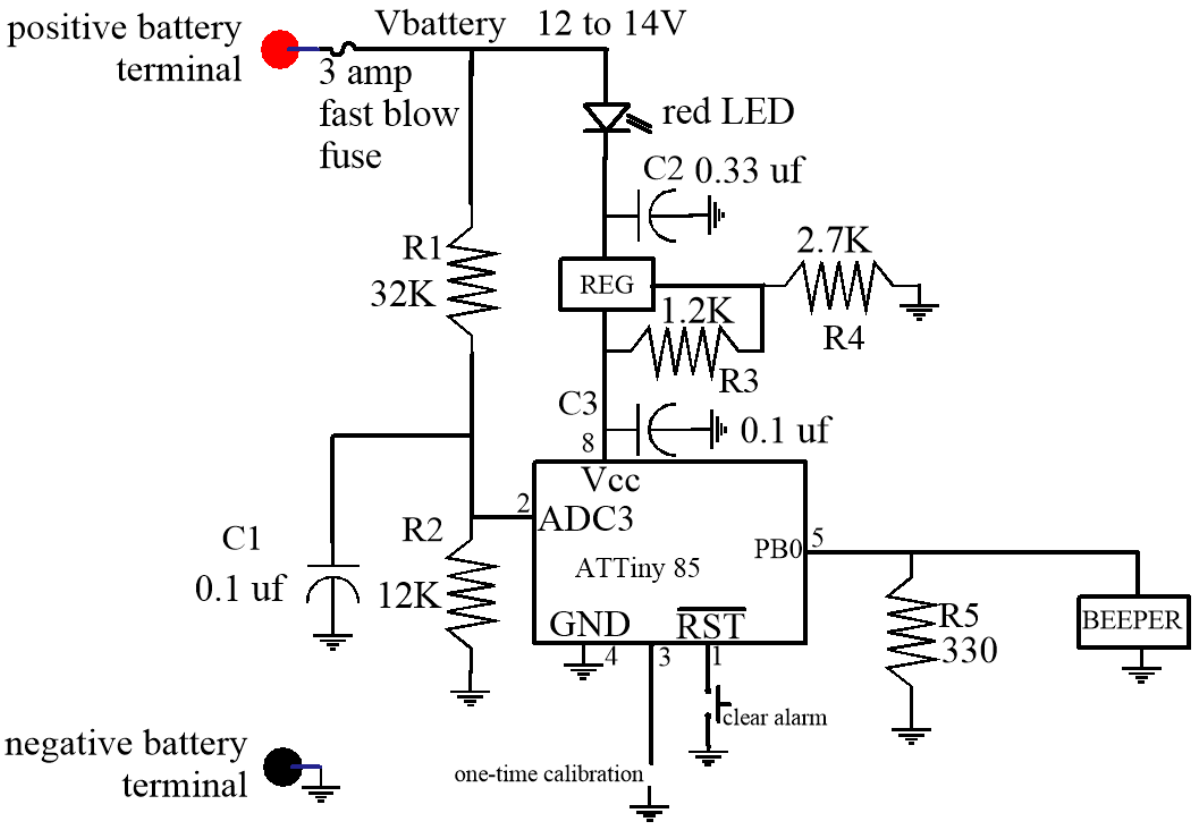
Hardware Design



The heart of the monitor is the ATTiny-85, in an 8-pin Dual In-line Package (DIP).



The ATTiny85 can be powered from a Vcc of 4V, and Vref can be set to Vcc. This gives me a nominal supply current drain of under 4 mA and an Analog-to-Digital Converter (ADC) with a max input of 4V.



Alarm Strategy

My visual output is a red LED in series with the voltage regulator. It glows as the ATTiny's typical 4 milliamperes flows through it. When pin 5 goes to logic 1, an additional 12 milliamperes flows, making the LED brighter. The contrast between these two brightness levels is evident. Dim means the processor is powered, and bright means there is an alarm.

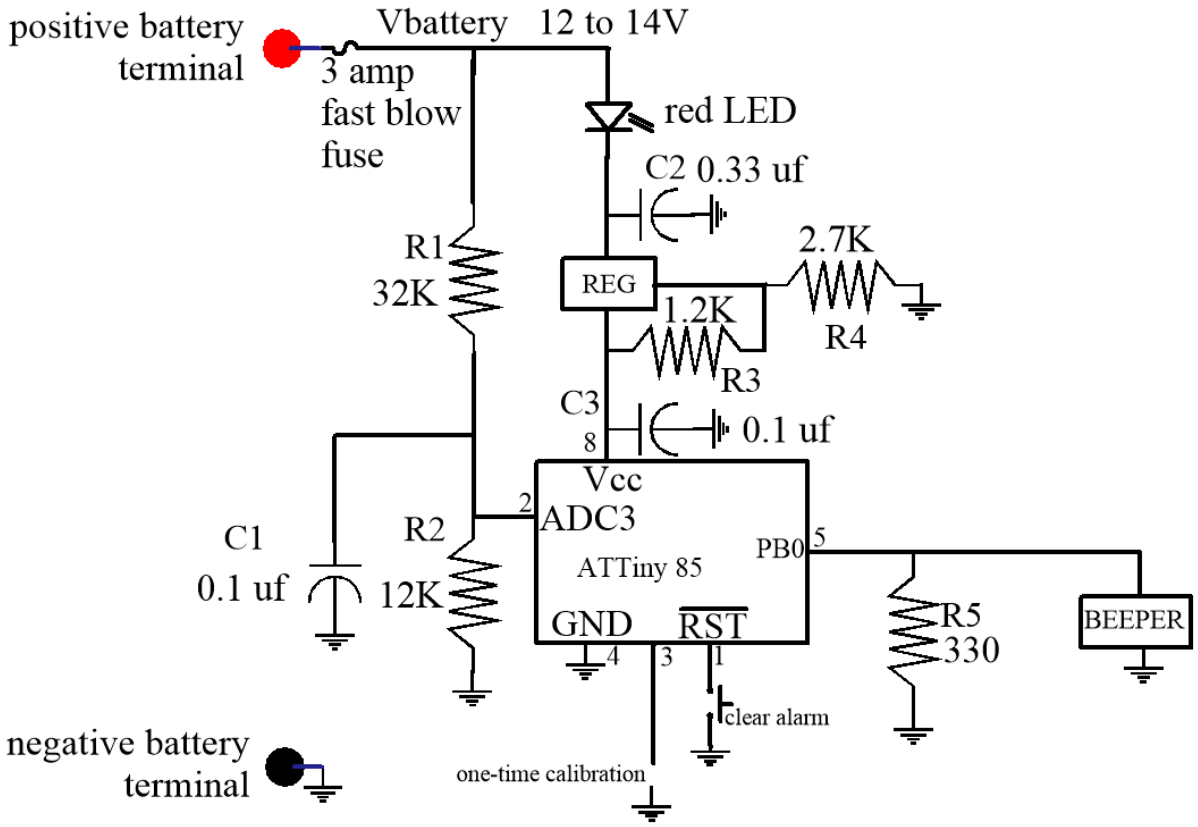
An alarm indicates it is time to perform a load test on the battery, as it may be near the end of its life. My experience has been that this condition does not constitute an emergency, just a warning. I check my various fluids under the hood each month, so checking for the flashing LED would be a small addition to the task.

For those who do not check their fluids monthly, the beeper would be more useful than the flashing LED. However, the beeper is large and costly.

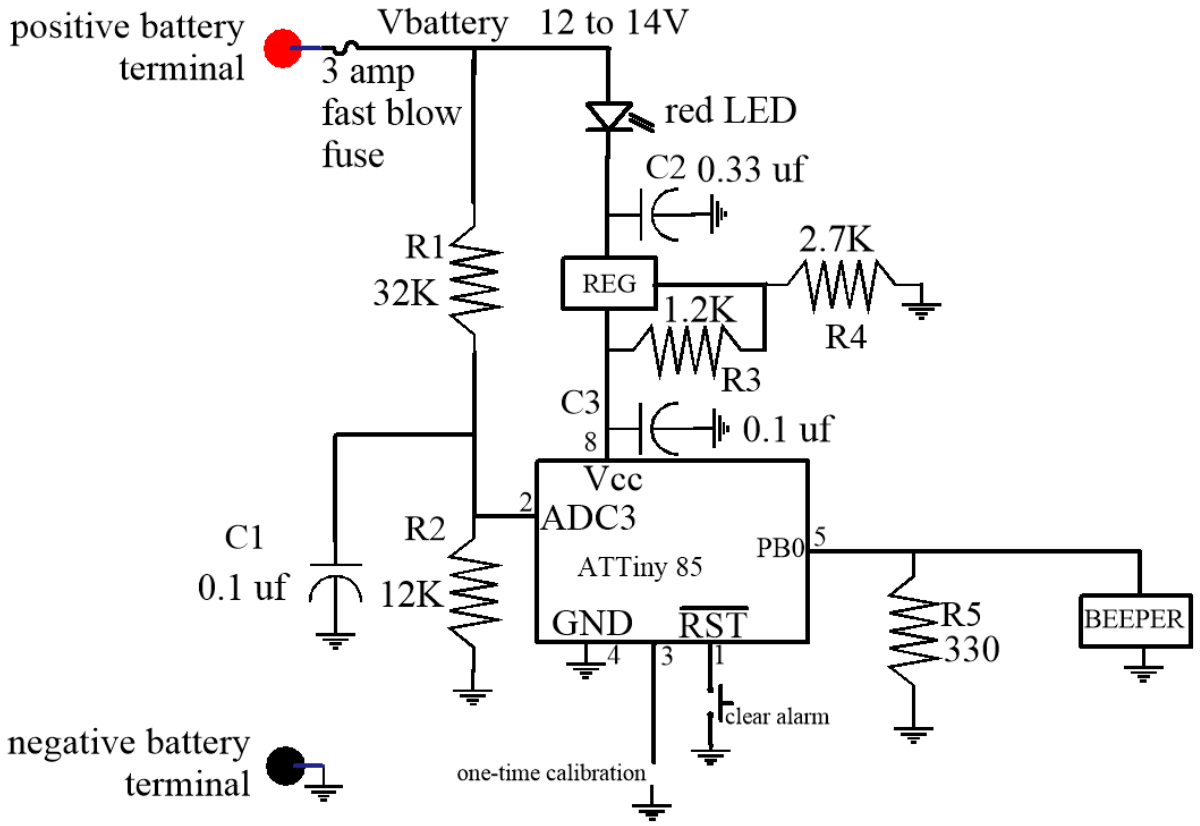
The final choice of output device is to be determined.

Circuit Design Choice

I am using a 3-terminal adjustable voltage regulator set up to output a nominal 4 volts. This voltage is connected to Vcc and serves as the ADC's reference voltage. My choice was based on having a large quantity of these devices, not on them being the lowest-cost solution. I could have used a 3-terminal 5-volt regulator, which would have saved two resistors and some space.



High-Level Circuit Description



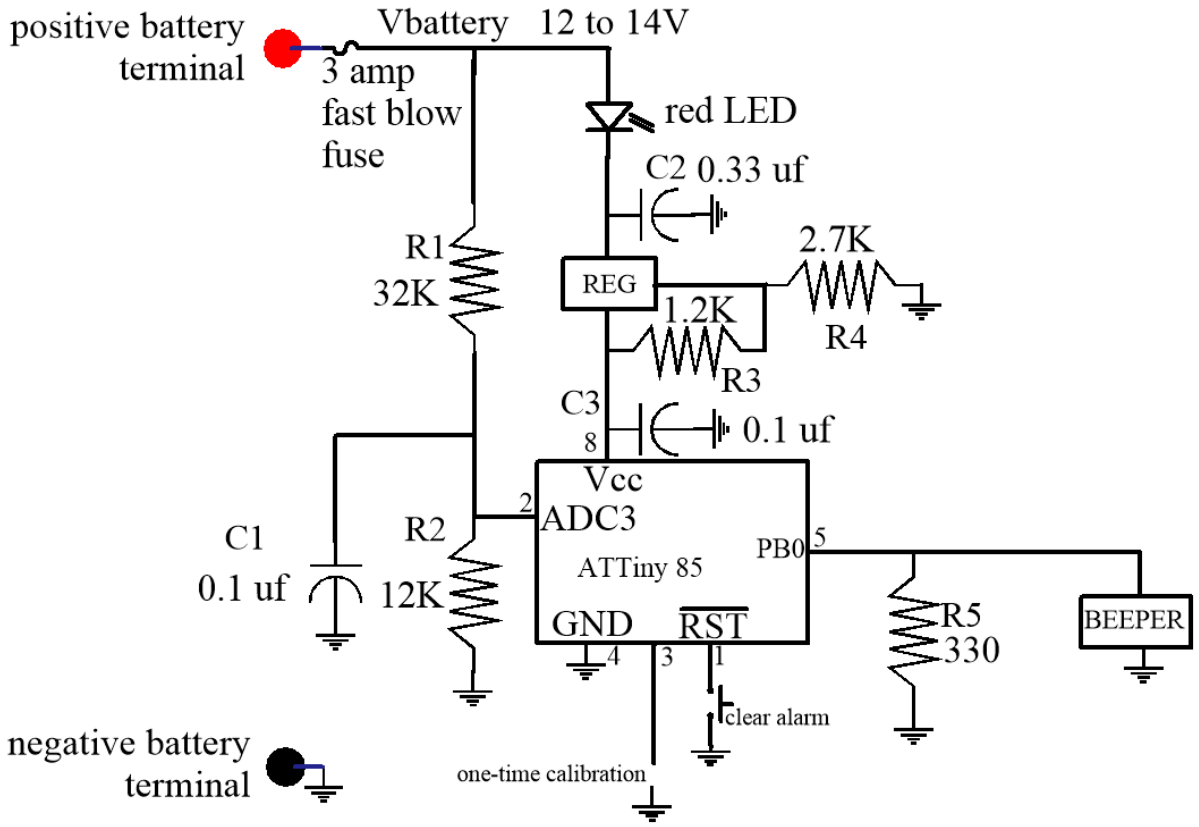
$V_{battery}$ is divided by R1 and R2 and fed into pin 2, which is the input to the Analog-to-Digital Converter.

When $V_{battery}$ is set to the threshold voltage[5], I momentarily connect pin 3 to ground. This action tells the software we are in calibration mode. The voltage on pin 2 is converted into a count and stored as our threshold. Note that this cancels the error in R1, R2, the ADC, and its reference voltage. The ATTiny contains a temperature sensor, which we use to record the ambient temperature during calibration.

During normal operation, while $V_{battery}$ is above the threshold, pin 5 is low. When $V_{battery}$ falls below the threshold, even briefly, the software pulses pin 5 up and down until the computer receives a reset.

[5] This is a one-time calibration, performed during fabrication.

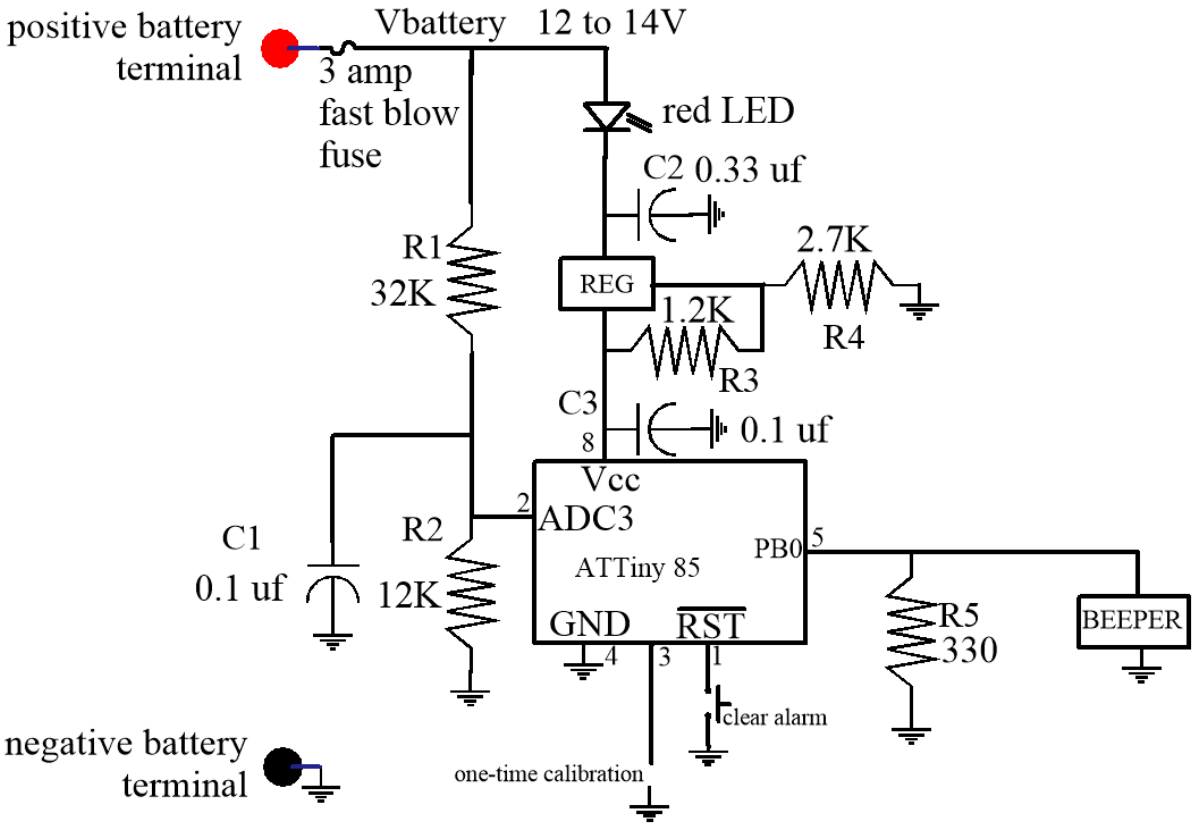
Nominally, the ATTiny draws 4 mA. This current flows from the battery, through the red LED, into the 3-terminal regulator, and into pin 8. It exists via pin 4 and returns via the negative battery terminal.



When pin 5 goes high, 12 mA flows through R5. This current is pulled through the regulator and therefore passes through the LED. This change in current from 4 to 17 mA is easily discerned as the red LED goes from dimmer to brighter.

When pin 5 goes high, the beeper also sounds.

Detailed Circuit Analysis



In general,

$$V_2 = \frac{R_2}{R_2 + R_1} \times Vbatt$$

$$V_2 = \frac{12K}{12K + 32K} \times Vbatt$$

$$V_2 = 0.2727 \times Vbatt$$

The ADC converts the input voltage to a count based on

$$count = \frac{v_2}{v_{ref}} \times 1024$$

Nominally, VREF is 4V. When Vbattery is at 12.000V, the nominal voltage on pin 2 is ($0.2727 \times 12.000V =$) 3.272V

$$count = \frac{3.272V}{4V} \times 1024$$

$$count = 838$$

When $V_{battery}$ is above 12.000V, the count will be above 838, and the software will continue to run the ADC and look for a count below 838.

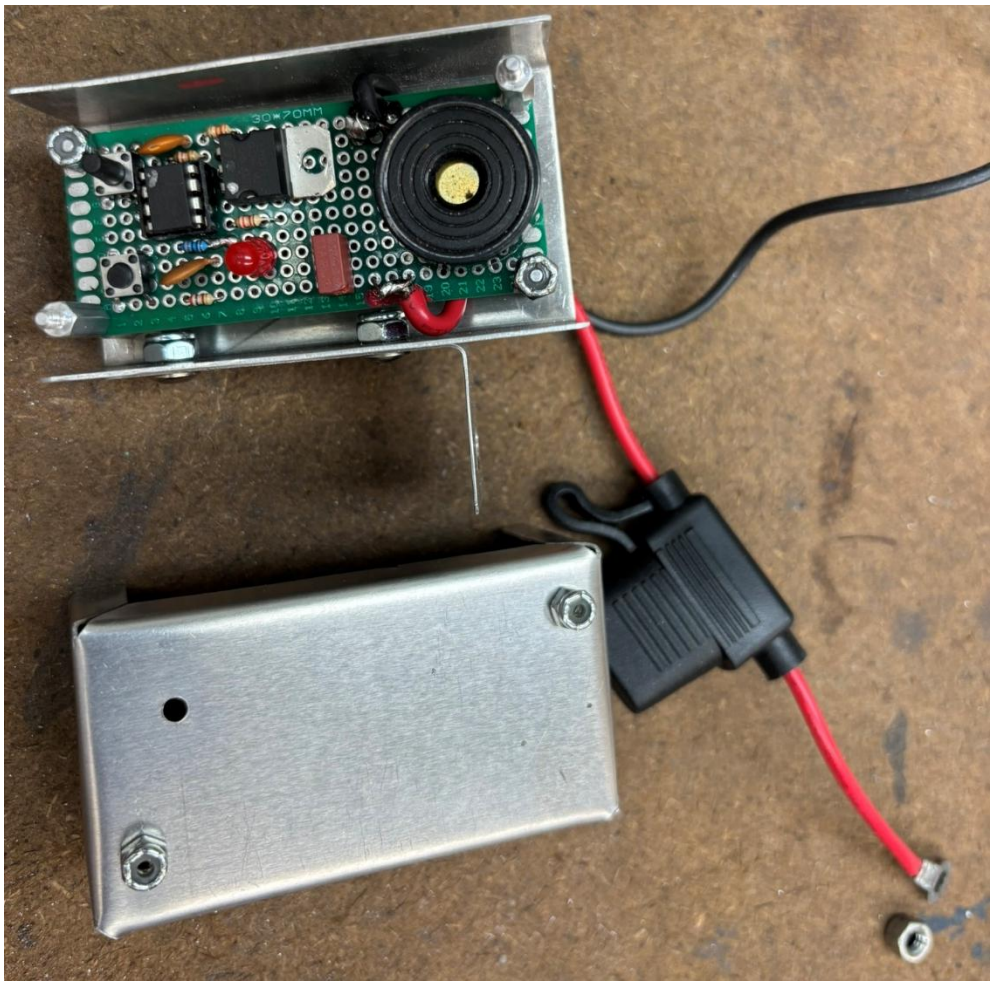
In the absolute worst case, given $V_{battery}$ equals 14V, V_2 gets up to 4.1V. This is above the range of the ADC but far below the safe input range for the ATTiny. If we changed to a 5V regulator, we would not reach the maximum ADC input voltage.

Parts List

- ATTiny 85 processor
- Red LED
- 32K, 12K, 1.2K, 2.7K, and 330 ohm 5% 0.1 watt
- (2) 0.1 uf ceramic
- 0.33 uf ceramic
- Pushbutton
- Piezoelectric beeper (optional)
- Circuit board
- Enclosure, rubber grommet for wires, mounting hardware[6]
- 3 amp fast blow automotive fuse in holder

[6] The hardware will experience vibration so use nylon lined lock nuts where possible and Loctite in other places.

Place the fuse as close to the positive battery terminal as possible to minimize the unfused wire length.



Software Overview

The code is written in C++ and compiled in the Arduino Integrated Development Environment (IDE). I set low-level ATTiny states using direct writes to hardware registers.

The code operates in two modes:

1. Calibration
2. Monitor battery voltage

Calibration is a one-time action performed during fabrication. A precision 12.000 volt bench supply powers the hardware. Then pin 3 is momentarily grounded. This causes the calibration software to run. The ADC count is stored in non-volatile memory along with a count representing the ambient temperature. Then the LED then flashes once.

After pressing the reset button briefly, the software enters battery monitor mode. It performs these tasks continuously

1. Read the ADC and store the count
2. Use the ADC to read the ambient temperature.
3. Calculate the temperature-compensated threshold using the stored threshold count, the stored calibration ambient temperature, and the current ambient temperature.
4. Compare the ADC reading to the temperature-compensated threshold.
    a. If the ADC reading is below the threshold, scan 5 more times and vote to see if the majority of readings are below the threshold.
        i. If the majority of readings are below the threshold, go into the alarm state.
        ii. If the majority of readings are above the threshold, start the procedure over.
    b. If the ADC reading exceeds the threshold, restart the process.

In the alarm state, we flash the LED between dim and bright, changing every ¼ second. Each time the LED becomes brighter, the beeper sounds. We leave this state when the reset button is pushed.

Hardware Registers

Some standard Arduino functions work as expected in the ATTiny. Other Arduino functions may exist, but I never found them, so I went back to the spec sheet and worked directly with its hardware registers. This approach requires a careful read of the spec sheet, which can be found at https://www.microchip.com/en-us/product/ATtiny85 and is 234 pages long. I didn't have to read all of it, but did have to understand enough to know what I needed to read. In the following, you can use the register's name as a search parameter to find more information in this document.

ADMUX

This register controls aspects of the ADC and a multiplexer that selects which input the ADC will measure.

When measuring the battery voltage, I set the following bits:

- Use V_{cc} instead of V_{ref} set bit 7 = 0, bit 6 = 0, bit 4 = 0
- To have the ADC output to be right-adjusted, set bit 5 = 0
- The right 4 bits are MUX[3:0] = 0011, which ties our ADC input to ADC3 and makes it a single-ended input. This works in conjunction with analogRead(ADCport_Byte) . ADC3 is physical pin 2, which is our divided-down battery voltage.

All of these bits are assigned to the constant I call `ADMUX_OperationalConfigurationByte`.

When measuring the temperature, the MUX is changed to a port that does not come out to a pin. This port ties to the on-chip temperature sensor.

- Use 1.1V as Vref (bit 7 = 1, bit 6 = 0, bit 4 = 0)
- To have the ADC output to be right-adjusted set bit 5 = 0
- The right 4 bits are MUX[3:0] = 1111: ADC4 is the temperature sensor input.

I assigned these bits to the constant `ADMUX_TemperatureConfigurationByte`.

CLKPR

This register controls the system clock's prescaler. By experience, I have found that the prescaler sometimes changes from x1 to x8, which slows system timing by a factor of 8. As a defensive measure, I set the prescaller to x1 in setup().

- Bit 7 is 1, and all other bits are 0
- Then I set bit 7 to 0, and that locks in the remaining bits.

EEPROM

The ATTiny contains a small quantity of non-volatile memory, EEPROM. My program uses four bytes.

When reading and writing to the EEPROM, we use these hardware registers:

- EECR is the EEPROM Control Register. I use it to tell the hardware that I want to write to the EEPROM, and it tells me whether the EEPROM is ready for this operation.
- EEPE is a status bit that says when you can access the EEPROM. It is located in the EECR
- EEARL is set to the LSB of the address. In my case, it is the address.
- EEARH is set to the MSB of the address. In my case, it is always 0.
- EEDR is where the read data is placed.

Fortunately, the spec sheet includes the code, in C++ needed to read and write to the EEPROM. Most of it makes sense.

Software Middle Level View

In this section, I assume the reader has minimal understanding of Arduino.

setup()

All code in setup() executes at power-up or reset. This is where I set up the environment, plus retrieve the data stored in the EEPROM.

loop()

While setup() only runs once, loop() executes all code within it until power is removed or the reset pin is pulled low.

The first function in `loop()` is `calibrationQ()`. The "Q" stands in place of "?" because "?" is an illegal character in a function name. It checks whether the calibration signal is active (pin 3 momentarily grounded). If this pin is low, a calibration is performed. This would be done only as the final step in the board's fabrication. Results are stored in the EEPROM.

The second function in loop() is `checkThreshold()`. It reads the battery voltage and acts on the results.

Software Low-Level View

See GitHub for the code: https://github.com/rgsparber/Chevy-Bolt-EV-electric-car-12-Volt-Battery-Monitor

What Next?

This project is protected under Creative Commons. You are free to copy, modify, and sell it with no obligation to me other than not removing my name from my design. If I were going into production, I would switch to surface-mount devices, look into ATtiny devices with less program storage, change to a TO-92 voltage regulator, and use a 3D-printed enclosure made of PETG. A double-sided board can easily support the circuit.